République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
École Supérieure en Informatique 8 Mai 1945 Sidi Bel Abbès

Master Thesis

Degree

Field: Computer Science
Specialty: ISI

Theme

Spatio-Temporal Foundation Model for Network Intrusion Detection Systems (NIDS) (Self-Supervised Zero-Day Detection via Hybrid Masked Transformers)

Presented by:
CHIALI Akila

Submission Date:
June 2026

In front of the jury composed of:

Dr. BENDAOUD Fayssal	Supervisor
Dr. Houari BOUMEDIENE	Examiner

Academic Year: 2025-2026

Acknowledgements

Abstract

Nowadays, the cybersecurity field is facing a significant dilemma caused by the exponential evolution of cyber threat vectors. The emergence of AI-driven intrusion techniques, has rendered traditional detection frameworks increasingly insufficient. Consequently, identifying malicious intent now requires more than simple pattern recognition; it demands a deep understanding of the underlying dynamics of network communication, moving beyond the mere detection of static signatures.

This thesis proposes a robust **Network Intrusion Detection System (NIDS)** built upon the complexity of packet flow analogies. By leveraging **Hybrid Masked Spatio-Temporal Transformers**, we redefine network traffic as a spatio-temporal volume. Unlike classical models, our approach utilizes self-supervised learning to capture the joint evolution of internal packet structures (spatial) and flow sequences (temporal).

Through this architecture, the model achieves a profound understanding of both historical attack patterns and "New" unobserved exploits. By mastering the fundamental "grammar" of the network, the proposed system provides not only a solution to detect known attacks through their signatures but also promises a scalable and adaptive resolution for **Zero-Day detection**, bridging the gap between high-capacity computational scaling and the complex nuances of modern cybersecurity.

Keywords: Spatio-Temporal Transformers, Masked Modeling, NIDS, Zero-Day Detection, Self-Supervised Learning..

Contents

List of Tables

List of Figures

List of Acronyms

Acronym	Definition
AI	Artificial Intelligence
ANN	Artificial Neural Network
APT	Advanced Persistent Threat
AUC	Area Under the Curve
BERT	Bidirectional Encoder Representations from Transformers
BiLSTM	Bidirectional Long Short-Term Memory
B5G	Beyond Fifth Generation
CNN	Convolutional Neural Network
DDoS	Distributed Denial of Service
DL	Deep Learning
FFN	Feed-Forward Network
FPR	False Positive Rate
GELU	Gaussian Error Linear Unit
HM-STT	Hybrid Masked Spatio-Temporal Transformer
IDS	Intrusion Detection System
IoT	Internet of Things
LSTM	Long Short-Term Memory
MAE	Masked Autoencoder
MFM	Masked Flow Modeling
MHA	Multi-Head Attention
ML	Machine Learning
NIDS	Network Intrusion Detection System
NLP	Natural Language Processing
PCAP	Packet Capture
PE	Positional Encoding
ReLU	Rectified Linear Unit
RNN	Recurrent Neural Network
SLAD	Scale Learning-based Deep Anomaly Detection
SSL	Self-Supervised Learning
STT	Spatio Temporal Transformer
ViT	Vision Transformer

Part I

Introduction

General Introduction

The integration of digital systems into every facet of modern life has created an attack surface of unprecedented scale. As the world moves towards hyper-connectivity through 5G and B5G networks, the complexity of cyber threats has evolved **simultaneously with these technological advancements** [1]. Malicious actors now employ polymorphic malware and techniques that **abuse legitimate system tools already present in the operating system to carry out attacks**, allowing their activities to blend seamlessly with normal network traffic [2]. In this adversarial environment, traditional signature-based Network Intrusion Detection Systems (NIDS) remain blind to Zero-Day attacks—exploits that target undisclosed vulnerabilities and possess no pre-existing signature [3].

The cybersecurity community has increasingly turned to Artificial Intelligence (AI) to close this gap. Deep Learning (DL) models have demonstrated superior performance in identifying malware variants compared to classical Machine Learning (ML). However, the dominant paradigm of Supervised Learning faces diminishing returns due to the bottleneck of labeled data [4] and "concept drift," where the statistical properties of **legitimate and harmless network traffic** shift over time, leading to significant model degradation [5].

This thesis argues that the solution lies in **Self-Supervised Learning (SSL)** and the development of **Foundation Models** for network traffic. Drawing inspiration from NLP and Computer Vision [6], [7], we propose that network traffic should be treated as a **Spatio-Temporal** signal. Traffic flows exhibit "spatial" structure within a packet's content and "temporal" dependencies across the sequence of packets [8]. Existing approaches have largely treated these dimensions in **isolation**, either converting flows to static 2D images (losing temporal granularity) **or** treating them as simple 1D sequences, often introducing unnecessary computational redundancy [9].

Architectures based on Spatio-Temporal Transformers (STT) demonstrate significantly superior performance compared to traditional recurrent prediction models across several application domains. Analyzing network traffic requires the simultaneous evaluation of spatial network topology and temporal packet flows, similar to tracking human joints [10] or heterogeneous vehicles in sequence data [11]. This research bridges these domains to build a resilient, adaptive defense mechanism for modern network infrastructures.

Motivation

Among the numerous studies demonstrating the effectiveness of Spatio-Temporal Transformers (STT), we focused on two fundamental architectures below, which serve as the foundation for our proposed Hybrid Masked Spatio-Temporal Transformer (HM-STT).

- In the first study [10] published in 2020 for Human Motion Prediction . the Spatio-Temporal Transformer is designed to model structured sequential data by separating spatial relationships from temporal dynamics. In many sequence prediction problems, it is necessary to capture both spatial dependencies, which describe the relationships between elements within the same time step, and temporal dependencies, which describe how these elements evolve over time.

  Traditional Recurrent Neural Networks (RNNs) process sequences sequentially and often suffer from exposure bias and error accumulation when predicting long sequences. Small prediction errors may propagate through time, leading to progressively degraded outputs.

  By jointly modeling spatial structure and temporal evolution, the ST-Transformer can effectively capture long-term dependencies while preserving structural consistency. Experimental results affirm that ST-Transformer model, achieved the best performances compared to other models in the 3D human motion prediction study on the AMASS dataset [12].

- Next study [11] the S2TNet framework utilizes STTs to predict the trajectories of heterogeneous traffic agents for highly dynamic environments (e.g., vehicles, pedestrians, cyclists). S2TNet relies on a spatial transformer to capture the complex topological interactions among all agents, moving beyond simple spatial neighborhood heuristics. Simultaneously, a temporal transformer enhances the modeling of dependencies to output future trajectories autoregressively. S2TNet reduces the Average Displacement Error compared to other baseline models, it proves that STTs are exceptionally capable of handling heterogeneous, dense, and non-holonomic interactions.

  The success of these architectures provides a strong theoretical basis for applying STTs to Intrusion Detection Systems (IDS). Network traffic is inherently heterogeneous and spatio-temporal, making it well-suited for decoupled attention mechanisms. The ST-Transformer demonstrates that exploiting spatial and temporal attention eliminates the error accumulation seen in RNNs when monitoring long sequences [10], which is essential for tracking prolonged Advanced Persistent Threats (APTs). By adapting these efficiency mechanisms, our proposed model can process **Large Scale** dense network in real time. Treating network logs as spatio-temporal signals allows the model to capture complexe signatures of zero-day attacks without relying on static 2D image conversions or isolated 1D sequential models.

Problem Statement

Current Machine Learning-based NIDS architectures face three critical challenges:

1. **Dependency on Labeled Data:** Supervised systems are brittle against "unknown unknowns."
2. **Inefficient Representation:** CNNs are biased towards local features, while RNNs struggle with long-range dependencies. Converting packet data into 2D images introduces artificial correlations [13].
3. **Lack of Generalization and adapatation :** Models trained on one network environment rarely transfer well to another.

Research Objectives

The primary objective is to develop a **Spatio-Temporal Foundation Model for NIDS** capable of detecting Zero-Day attacks without reliance on attack-specific training labels. Sub-objectives include:

- **Tubelet Representation:** Transforming raw packets into 3D spatio-temporal "tubelets" [14].
- **Hybrid Masked Architecture:** Implementing a Hybrid Masked Spatio-Temporal Transformer (HM-STT) utilizing a masked reconstruction pretext task.
- **Zero-Day Evaluation:** Benchmarking against state-of-the-art frameworks.
- **Accuracy and Efficiency:** Maximizing detection accuracy, minimizing prediction error, and reducing computational complexity for real-time network monitoring.

Part II

Background

Chapter 1

Network Security and the Evolving Threat Landscape

1.1 Architectural Framework of Intrusion Detection Systems

An Intrusion Detection System (IDS) is not a monolithic entity but a modular architecture designed to collect, process, and analyze data in real-time. According to the **Common Intrusion Detection Framework (CIDF)**, a robust IDS consists of four primary functional components:

1. **The Sensor (E-box):** Responsible for the ingestion of raw network telemetry. In the context of NIDS, this involves capturing traffic via passive monitoring (e.g., Port Mirroring).
2. **The Analyzer (A-box):** The intelligence layer where the detection logic is applied. This thesis focuses on optimizing this component using a **Hybrid Masked Transformer** to identify anomalous spatio-temporal patterns.
3. **The Storage (D-box):** A repository for configuration data, learned behavioral baselines, and historical logs.
4. **The Manager (R-box):** The interface that translates the analyzer's findings into human-readable alerts or automated defensive actions.

1.1.1 Architectural Placement

The effectiveness of the IDS is highly dependent on its placement within the network topology. While **Host-based IDS (HIDS)** provide granular visibility into endpoint activities, **Network-based IDS (NIDS)** offer a broader view of the communication fabric, making them essential for detecting multi-stage APTs and Zero-Day exfiltration attempts [15].

1.1.2 Intrusion Detection Paradigms

Intrusion Detection Systems (IDS) are categorized by their underlying detection logic. Selecting the appropriate paradigm is a trade-off between computational overhead and the ability to generalize to unseen threats.

- **Signature-Based Detection (Knowledge-Based):** This deterministic approach relies on a database of predefined patterns or "fingerprints" of known malicious activity. While highly

effective for preventing recurring threats with near-zero false positive rates, it remains fundamentally **blind to Zero-Day attacks** and polymorphic malware that lacks a prior signature .

- **Anomaly-Based Detection (Behavior-Based):** Instead of looking for "bad" patterns, this paradigm learns the statistical baseline of "normal" network behavior. Any significant deviation from this baseline is flagged as an intrusion. This is the primary domain of **Self-Supervised Learning**, as it allows for the detection of novel exploits, though it requires careful tuning to manage higher False Positive Rates (FPR) .
- **Specification-Based Detection:** This method defines a set of formal constraints and "correct" protocol behaviors (e.g., following RFC standards for TCP/IP). It flags any activity that violates these logical specifications. While robust, it is manually intensive to define and maintain for complex, evolving protocols .
- **Hybrid Detection:** Modern architectures, such as the one proposed in this thesis, often combine these methods. By integrating the speed of signature matching with the generalizability of anomaly-based transformers, hybrid systems maximize detection coverage while minimizing false alarms .

1.1.3 Categorization of Network Attack Vectors

To evaluate the proposed model, we categorize the threats present in the dataset into five primary functional classes:

Table 1.1: Taxonomy of Common Network Attacks in NIDS Datasets.

Category	Examples	Description
Denial of Service	DoS, DDoS, Slowloris	Attempts to exhaust system resources (CPU, bandwidth) to render services unavailable.
Exfiltration	Data Theft, Tunneling	The unauthorized transfer of sensitive data from the target network to an external C2 server.
Brute Force	SSH-BF, FTP-BF	Systematic attempts to crack credentials through automated trial-and-error.
Web-Based	SQLi, XSS, Brute Force	Exploiting vulnerabilities in web applications to gain unauthorized access or inject code.
Infiltration	Backdoors, Exploits	Leveraging software vulnerabilities to gain a foothold inside a secure internal network.

1.1.4 Zero-Day Exploits and Polymorphic Malware

A critical vulnerability in modern infrastructures is the rise of Zero-Day exploits attacks, that target software vulnerabilities completely unknown to the vendor and security community [16]. Because no patch or historical signature exists for a zero-day exploit, legacy systems are fundamentally blind to them.

Compounding this issue is the use of polymorphic and metamorphic malware [17] . Attackers utilize automated generation tools that constantly alter the identifiable footprint (hashes, byte structures, and command-and-control IP addresses) of the malware payload. Consequently, organizations must implement robust, defense-in-depth strategies. Within this hyper-connected ecosystem, continuous monitoring and automated threat detection serve as the foundational backbone [18].

1.2 The Escalation of Network Complexity

The architectural complexity of modern digital infrastructures has expanded exponentially, driven by the proliferation of the Internet of Things (IoT), the deployment of Beyond Fifth Generation (B5G) communication networks, and the decentralization inherent in edge computing paradigms [19]. This rapid expansion has elevated network security from a secondary operational concern to a critical imperative, as the attack surface has fundamentally expended [20]. With billions of inherently insecure IoT devices continuously transmitting data, the volume, velocity, and variety of network traffic have made human-led and traditionnal network monitoring mathematically impossible.

1.2.1 Advanced Persistent Threats (APTs) and Lateral Movement

Modern adversaries no longer rely on singular, destructive events. Instead, the threat landscape is dominated by Advanced Persistent Threats (APTs) stealthy , continuous computer network attack processes orchestrated by highly resourced groups. APTs are characterized by their multi-stage nature: initial compromise, establishing a foothold, lateral movement across the network, and eventual data exfiltration [21].

Because APTs operate over extended temporal windows (often lying dormant for months), traditional security systems that evaluate network packets as isolated, instant events fail to detect the global malicious narrative. Detecting lateral movement requires a system capable of linking a suspicious login on one server (spatial feature) to anomalous data transfer days later (temporal feature)[22].

1.3 The Failure of Traditional NIDS

As networks grow in scale, traditional perimeter defenses such as static firewalls and legacy Network Intrusion Detection Systems (NIDS) are no longer sufficient [23][24].

1.3.1 Signature-Based vs. Anomaly-Based Detection

Historically, NIDS relied upon signature-based detection mechanisms (e.g., Snort, Suricata) [25]. These systems operate by cross-referencing incoming packet payloads against a static database of known malware hashes. While computationally efficient—often operating with $\mathcal{O}(1)$ or $\mathcal{O}(n)$ complexity for pattern matching—this deterministic approach is structurally incapable of identifying zero-day vulnerabilities or polymorphic variations.

To address the limitations of static signature matching, modern intrusion detection must shift toward anomaly-based deep learning paradigms. Introduced conceptually by Denning [26], anomaly detection builds a baseline of "normal" network behavior and flags deviations. While anomaly detection has the theoretical capability to identify Zero-Day exploits, it historically suffers from high false-positive rates due to the unpredictable, shifting nature of legitimate network traffic.

1.3.2 The Challenge of Encrypted Traffic

Historically, NIDS relied heavily on Deep Packet Inspection (DPI) to parse the payload and search for malicious command strings. However, the ever-present adoption of cryptographic protocols (like TLS 1.3) has rendered the payload completely opaque to intermediate network sensors. Without the decryption keys, DPI is obsolete [27].

To maintain visibility, modern NIDS must pivot away from plaintext payload inspection. They must rely exclusively on analyzing the structural, spatial, and temporal patterns of the encrypted

byte sequences, the packet headers, and the communication frequencies , necessitating the use of advanced deep representation learning.

1.3.3 The Base Rate Fallacy

The primary mathematical challenge in anomaly-based NIDS is the Base Rate Fallacy, formalized by Axelsson [28] as :

$$P(I|A) = \frac{P(A|I)P(I)}{P(A|I)P(I) + P(A|\neg I)P(\neg I)}$$

Where $P(I)$ is the base rate of intrusions, $P(\neg I)$ is the base rate of legitimate traffic, $P(A|I)$ is the true positive rate, and $P(A|\neg I)$ is the false positive rate. Because malicious packets represent an extremely small fraction of total global network traffic, even a highly accurate model with a apparent negligible false-positive rate (e.g, 1%) will generate an overwhelming absolute number of false alarms. This phenomenon causes "alert fatigue," rendering the NIDS practically unusable. Consequently, modern NIDS require foundation models capable of learning highly precise, to push the false-positive rate as close to zero as mathematically possible.

1.4 Network Traffic Analysis Granularity

To effectively detect the threats, it is necessary to establish the granularity at which network traffic is analyzed. Traditional systems often fail because they analyze data at the incorrect abstraction layer.

1.4.1 Packet-Level vs. Flow-Level Analysis

Network traffic can be fundamentally evaluated through two primary levels: individual packets or aggregated flows.

- **Packet-Level Analysis (Deep Packet Inspection):** This approach examines the raw payload and headers of every single packet traversing the network. While it provides granular visibility into exploit signatures, it is highly computationally expensive, breaks down completely in the presence of encrypted traffic (e.g, TLS 1.3), and lacks context regarding the state of the connection.
- **Flow-Level Analysis:** A network flow aggregates a sequence of packets sharing a common 5-tuple: Source IP, Destination IP, Source Port, Destination Port, and Protocol (TCP/UDP), captured over a predefined temporal window. Flow analysis records statistical metadata such as packet counts, byte volumes, inter-arrival times, and duration.

Modern Advanced Persistent Threats (APTs) are rarely identifiable via a single packet. Therefore, modeling network data requires a shift toward flow-level aggregation. Treating these flows as a continuous spatio-temporal signal allows deep learning models to observe the macro-behavior of an attack over time, rather than getting lost in the micro-noise of isolated, encrypted packets.

1.5 The Anatomy of Advanced Attacks

To understand why temporal modeling is mandatory for modern NIDS, cyber intrusions must be viewed through formalized, multi-stage frameworks rather than as isolated anomalies.

1.5.1 The Cyber Kill Chain Framework

The Lockheed Martin Cyber Kill Chain [29] models a cyberattack as a strict sequence of operational phases:

1. **Reconnaissance:** The adversary gathers intelligence on the target architecture.
2. **Weaponization:** Creating a adapted malware payload.
3. **Delivery:** Transmitting the payload via vectors like phishing or drive-by downloads.
4. **Exploitation:** Triggering the vulnerability in the target system.
5. **Installation:** Establishing a persistent backdoor.
6. **Command and Control (C2):** Opening a bidirectional communications channel to the attacker's server.
7. **Actions on Objectives:** Lateral movement, privilege escalation, or data exfiltration.

Traditional systems evaluate network packets as instantaneous events, missing the correlation between Phase 1 (reconnaissance port scans) on day one, and Phase 7 (exfiltration) weeks later. A Spatio-Temporal architecture is specifically designed to maintain this long-range temporal state, effectively mapping the entire kill chain as a single, extended anomaly.

1.6 Adversarial Evasion and AI Blindspots

As defense mechanisms have integrated artificial intelligence, attackers have simultaneously adopted Adversarial Machine Learning to actively bypass AI-driven boundaries [30].

Through adversarial perturbation, adversaries make minimal, constrained feature changes to their network traffic. These evasion techniques include:

- **Packet Timing Manipulation:** Altering the inter-arrival times of packets to disguise automated C2 beacons as infrequent, human-generated web traffic.
- **Dummy Byte Injection:** Padding payloads with randomized noise to shift statistical distributions (like byte entropy) away from known malware profiles.
- **IP Spoofing and Fragmentation:** Breaking malicious payloads across multiple, out-of-order packets to evade spatial signature matching.

These perturbations preserve the semantic validity of the exploit while mathematically pushing the data outside a discriminative model's learned decision boundary. This underscores the vital need for generative architectures (like Masked Autoencoders) that can ignore adversarial noise and reconstruct the underlying structural truth.

Chapter 2

Machine/Deep Learning: The Path to Spatio-Temporal Transformers

2.1 Evolution of Neural Networks in Cybersecurity

The transition from manual signature writing to automated threat detection naturally adopted traditional deep learning models. However, evaluating these foundational building blocks reveals why the field is currently pivoting heavily toward Spatio-Temporal Transformers (STTs).

2.2 Recurrent Neural Networks (RNNs) and Temporal Limitations

RNNs and LSTMs were initially utilized for network traffic because they maintain a hidden state h_t to model sequential data (e.g, a flow of packets). While effective for short sequences, they process data sequentially and suffer from the Vanishing Gradient Problem. In the context of cybersecurity, an APT might initiate a connection on day one and exfiltrate data on day thirty. Standard RNNs lack the capacity to capture these long-range dependencies without severe error accumulation.

2.3 Convolutional Neural Networks (CNNs) and Spatial Limitations

To overcome the slowness of RNNs, researchers attempted to convert network packets into 2D grayscale images, processing them with CNNs. CNNs utilize kernels to extract local spatial features. While highly effective for actual images, applying CNNs to network data assumes that adjacent bytes are highly correlated (like pixels). This forces an artificial spatial relationship onto network features that may not exist, while simultaneously completely ignoring the temporal evolution of the traffic flow.

2.4 Transformers and Attention Mechanisms

To resolve the bottlenecks of RNNs and the artificial constraints of CNNs, the focus shifted to the Transformer architecture, introduced in the seminal paper **"Attention Is All You Need" (2017)** [6]. The core idea behind transformers is the self-attention mechanism, which enables the model to weigh different parts of the input sequence globally, completely bypassing sequential processing constraints.

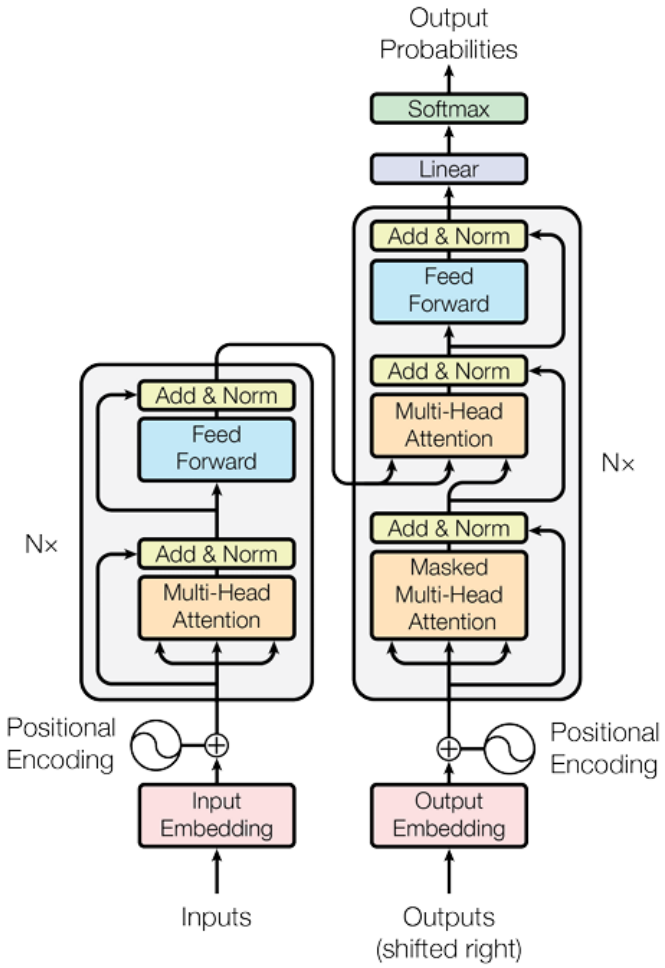

Figure 2.1: Model General Architecture [6].

2.4.1 Encoder and Decoder Stacks

Encoder

The encoder consists of $N = 6$ identical layers. Each layer contains two sub-layers:

- Multi-head self-attention mechanism
- Position-wise fully connected feed-forward network (FFN)

Each sub-layer is wrapped with a residual connection followed by layer normalization:

$$\text{LayerNorm}(x + \text{Sublayer}(x)) \tag{2.1}$$

All sub-layers and embedding layers produce outputs of dimension $d_{\text{model}} = 512$.

Decoder

The decoder also consists of $N = 6$ identical layers. Each layer contains three sub-layers:

- Masked multi-head self-attention
- Encoder-decoder attention
- Position-wise feed-forward network

Note on Hyperparameter Configuration: It is critical to note that dimensions such as $N = 6$, $d_{\text{model}} = 512$, and $h = 8$ represent the baseline configuration established by Vaswani et al. [6] for NLP. In modern architectural design, particularly for Spatio-Temporal NIDS, these values are treated as tunable hyperparameters rather than fixed constraints, optimized for network traffic features and other requirements.

2.4.2 Scaled Dot-Product Attention

The attention function operates on queries Q, keys K, and values V:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2.2}$$

Scaling by $\sqrt{d_k}$ prevents large dot products from pushing the softmax into regions with very small gradients.

2.4.3 Multi-Head Attention and Positional Encoding

Instead of performing a single attention function, queries, keys, and values are linearly projected h times:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O \tag{2.3}$$

Because the Transformer has no recurrence or convolution, positional information must be added to the embeddings using sinusoidal functions so the model can capture the sequential order of network packets.

2.5 Vision Transformers (ViT) as a Stepping Stone

While standard Transformers excelled at NLP, applying them to multidimensional matrices required the **Vision Transformer (ViT)** [31]. ViT challenges the dominance of CNNs by treating 2D matrices as sequences of patches.

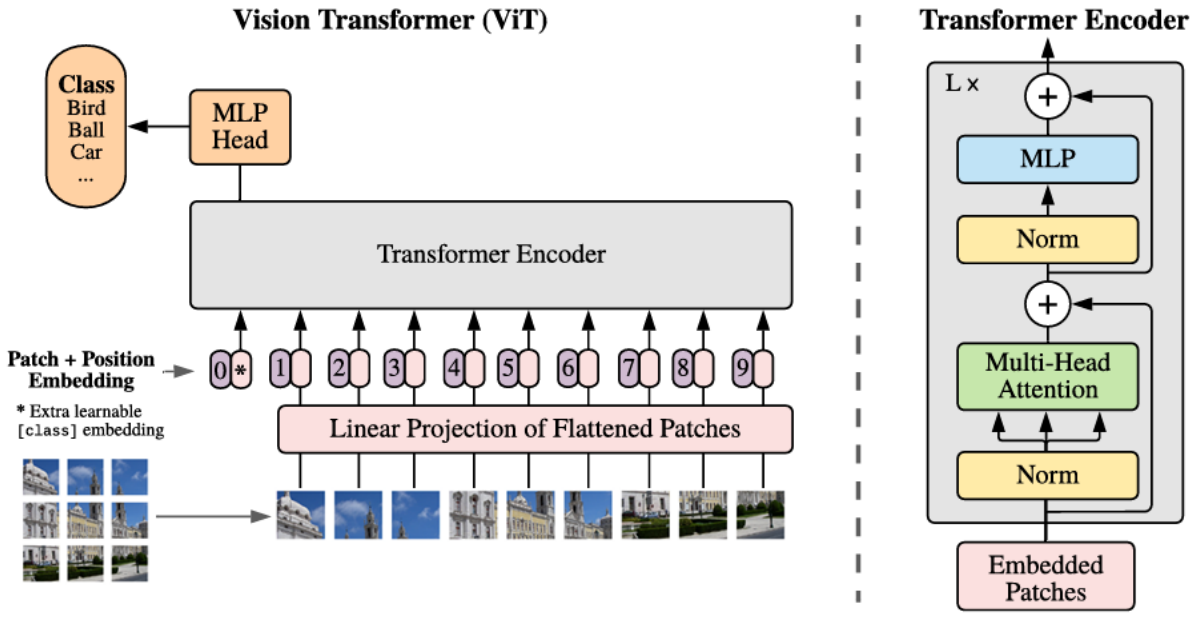

Figure 2.2: Model overview [31] .

The ViT model divides an input grid into non-overlapping 2D patches, flattening them into a sequence length $N = \frac{H \times W}{P^2}$. Similar to BERT [32], a `[class]` token is prepended to serve as the global representation. While ViT proves that self-attention can handle spatial matrices, adapting this strictly to 2D network anomaly detection still fundamentally ignores the temporal dimension of evolving packet flows. This limitation necessitates the evolution into Spatio-Temporal architectures.

2.6 Self-Supervised Learning and Masked Autoencoders

To solve the dependency on labeled zero-day attack data, we turn to Self-Supervised Learning (SSL). SSL formulates a "pretext task" where the supervision signal is generated automatically from unlabeled data [33].

The success of Self-Supervised Learning (SSL) in Natural Language Processing was largely driven by Masked Language Modeling (MLM), popularized by architectures like BERT [32]. Similarly, Computer Vision experienced a paradigm shift through Masked Image Modeling (MIM) via Masked Autoencoders (MAE) [7]

The Masked Autoencoder is a highly efficient architecture for generative SSL. It relies on an asymmetric design:

- **Asymmetric Encoder:** The input sequence is heavily masked (around 70%). The encoder processes only the small subset of visible tokens, significantly reducing computational overhead.
- **Lightweight Decoder:** The decoder reconstructs the original signal from the latent representations and learned "mask tokens."

The optimization objective minimizes the Mean Squared Error (MSE) strictly on the masked tokens:

$$\mathcal{L}_{MSE} = \frac{1}{|M|} \sum_{i \in M} \|x_i - \hat{x}_i\|^2 \tag{2.4}$$

This generative self-supervision is highly effective for NIDS. Because the model is trained exclusively on normal traffic, any anomalous packet sequence will violate learned correlations, resulting in a higher MSE anomaly score.

2.7 Masked Flow Modeling (MFM)

MFM was designed to address the inefficiencies and redundancies in spatial-domain masking and to leverage the unique properties of the frequency domain for more effective representation learning [34].

MFM masks and predicts in the frequency domain, where different frequency components capture different aspects of the data (e.g, low frequencies for global structure, high frequencies for fine details). This approach reduces redundancy and creates a more meaningful self-supervised task.

2.7.1 The Mechanics of MFM

In Masked frequency Modeling, a continuous network flow (comprising a sequence of packets, temporal features, or spatio-temporal tubelets) is divided into discrete tokens. During the pre-training phase.

The architecture's encoder processes exclusively the unmasked (visible) tokens, extracting deep contextual representations. Subsequently, the decoder attempts to predict or reconstruct the original, uncorrupted values of the masked tokens based purely on the bidirectional spatial and temporal context of the visible tokens [34].

Mathematically, given a flow sequence $X = \{x_1, x_2, \ldots, x_T\}$, a subset of indices M is chosen for masking. The objective is to minimize the reconstruction loss $\mathcal{L}_{\text{MFM}}$ strictly over the masked tokens:

$$\mathcal{L}_{\text{MFM}} = \sum_{i \in M} \text{Loss}(x_i, \hat{x}_i) \tag{2.5}$$

where $\hat{x}_i$ is the predicted token reconstructed by the decoder. For continuous network features, this loss function is typically expressed as the Mean Squared Error (MSE).

2.7.2 MFM as an Anomaly Detection Trigger

MFM is well suited for Network Intrusion Detection Systems (NIDS). Traditional supervised learning relies heavily on labeled attack data, creating inherent vulnerabilities to zero-day exploits. By employing MFM as a pretext task, the Transformer model learns the intrinsic syntax, structural dependencies, and temporal dynamics of purely *benign* network traffic without requiring human annotation.

When deployed for anomaly detection, the MFM framework calculates the reconstruction error of incoming, real-time traffic. Legitimate flows will closely match the learned syntax, yielding a low reconstruction error. Conversely, malicious traffic—such as Advanced Persistent Threats (APTs) or zero-day malware—inherently violates the learned temporal and spatial correlations of benign traffic. The decoder will fail to reconstruct these anomalous sequences, resulting in a severe spike in the Mean Squared Error. This reconstruction failure serves as a highly accurate, dynamic intrusion alert.

2.8 Spatio-Temporal Transformers for Network Topologies

A Spatio-Temporal Transformer, is a deep learning architecture designed to model data that evolves across both space and time. Its main objective is to learn relationships between entities observed at different positions or feature locations and at different time steps. Unlike classical recurrent models, which process sequences step by step, transformers rely on self-attention to capture short-range and long-range dependencies in parallel. This makes them especially suitable for complex multivariate sequences in which relevant interactions may occur across distant times or across multiple components of the input.

In a general setting, the input can be represented as a sequence of structured observations:

$$X = \{x_1, x_2, \ldots, x_T\}$$

where each x_t is not simply a scalar but a multidimensional observation describing the system at time t. In many applications, each x_t may itself contain several spatial components, feature channels, patches, nodes, sensors, or tokens. The purpose of the STT is to jointly model:

$$\text{spatial relationships} \quad + \quad \text{temporal relationships}$$

Standard 2D image models (like ViT) ignore temporal relations, making them unsuitable for continuous sequence predictions. Conversely, sequence models ignore topological structure. By extending into 3D architectures, we represent collected network data as a 3D volume (cube or cuboid), where spatial network features form the 2D grid, and time serves as the third dimension.

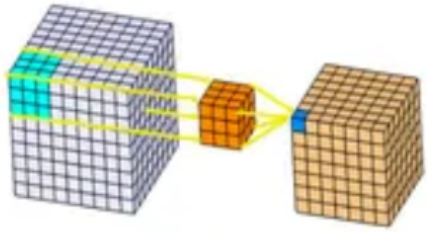

Figure 2.3: Representation of the network data in 3D

However the term **spatial** does not always mean physical 2D or 3D space. More generally, it refers to the structure internal to each observation at a given time step. Depending on the application:

- image patches in computer vision
- body joints in human motion modeling -
- sensors in IoT data
- graph nodes in traffic networks

Thus, the spatial dimension represents how components within the same time step relate to one another.

The **temporal** dimension represents how the observations evolve over time. It captures:

- sequential order
- short-term transitions
- long-term dependencies
- recurring patterns
- abrupt changes
- trends and periodicity

In other words, temporal modeling analyses how the current state depend on past and future states.

2.8.1 STT Concept

The central concept of an STT is to first encode each input element into a latent representation and then use self-attention to allow each token to interact with all others. This lets the model learn which parts of the spatio-temporal sequence are relevant for reconstructing, predicting, or classifying the data.

If the input sequence is tokenized into embeddings:

$$Z = \{z_1, z_2, \ldots, z_N\}, \quad z_i \in \mathbb{R}^d$$

then self-attention computes contextualized representations by comparing tokens through query, key, and value projections:

$$Q = ZW_Q, \quad K = ZW_K, \quad V = ZW_V$$

and then:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

This operation allows each token to attend to all other tokens, thereby learning both local and global relationships.

2.9 Spatio-Temporal Transformer (STT) Foundations

2.9.1 Generalized Architecture

A robust STT architecture functions as a multi-stage pipeline designed to map raw, time-varying signals into a structured latent space. The generalized framework consists of five primary modules:

1. **Input Tokenization:** The raw input is decomposed into discrete tokens. Depending on the domain, a token may represent a spatial patch (Computer Vision), a sensor vector (IoT), or a single packet's feature vector (Cybersecurity).

2. **Latent Projection (Embedding Layer):** Each token x_i is projected into a high-dimensional latent space d_{model} via a learnable linear transformation:

$$e_i = \mathbf{W}_{emb}x_i + \mathbf{b}_{emb} \tag{2.6}$$

   In other words to process this 3D volume, the model extracts "tubelets" small 3D blocks of data spanning across both the spatial network features and temporal time-windows.

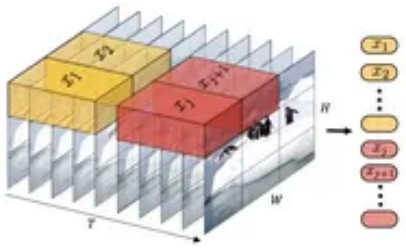

Figure 2.4: Visualisation of the tubelet embedding

   The tubelet is flattened into a vector representation $z \in \mathbb{R}^D$, making volumetric spatio-temporal information strictly compatible with the Transformer architecture.

3. **Encoding Order (Positional & Temporal):** As Transformers are permutation-invariant, we inject structural information by adding positional encodings p_i:

$$h_i = e_i + p_i \tag{2.7}$$

   This ensures the model distinguishes between "Feature A at Time 1" and "Feature A at Time 10."

4. **Transformer Encoder Blocks:** A stack of L layers, each containing *Multi-Head Self-Attention (MHSA)* and *Feed-Forward Networks (FFN)*, processes the tokens in parallel to capture global dependencies.

5. **Task-Specific Head:** The final hidden states are passed to a specialized head for objectives such as **Anomaly Detection**, **Traffic Forecasting**, or **Masked Reconstruction**.

[Image of Spatio-Temporal Transformer (STT) architecture diagram]

2.9.2 Spatial and Temporal Attention Strategies

The efficiency of an STT depends heavily on how it reconciles the two dimensions of data:

- **Joint Attention:** A single self-attention mechanism processes all spatio-temporal tokens simultaneously. While exhaustive, it suffers from $O((T \times S)^2)$ complexity.
- **Factorized Attention:** The model alternates between spatial attention (within a time step) and temporal attention (across time steps). This reduces computational overhead while maintaining structural hierarchy [14].
- **Hierarchical Attention:** Local patterns are learned first and subsequently aggregated into global representations, mimicking the receptive field expansion of CNNs.

2.9.3 Hyperparameters

Hyperparameters are external settings that are defined before training begins and control how the model learns . The prefix 'hyper-' indicates that these are 'top-level' parameters that govern the learning process itself rather than being part of the resulting model.

Key characteristics of hyperparameters:

- External settings defined before training that control how the model learns.
- Not learned by the model, but chosen manually or through optimization techniques.
- External to the model and not part of the final trained model.
- Critical for model performance and generalization ability.

Examples of hyperparameters:

- **Learning rate:** Controls how fast parameters are updated during training, affecting convergence speed and stability.
- **Number of epochs:** Determines how many times the model sees the full dataset during trainin.
- **Batch size:** Number of samples processed before updating parameters, influencing training efficiency and memory usage .
- **Number of layers/neurons in neural networks:** Architecture decisions that determine model capacity and complexity .
- **Regularization strength (e.g., L1/L2):** Controls overfitting by penalizing large parameter values .

- **Number of trees/depth in Random Forest or Gradient Boosting:** Ensemble specific parameters affecting model complexity and performance .
- **Choice of activation function:** Determines the non-linear transformation applied to neuron outputs. Examples include ReLU, Sigmoid, Tanh, Leaky ReLU, ELU, and Swish .
- **Choice of optimizer:** Determines the algorithm used to update model parameters during training. Examples include SGD, Adam, RMSprop, AdaGrad, and Nadam .

The distinction between parameters and hyperparameters is fundamental to machine learning: while parameters define what the model has learned, hyperparameters define how the learning process itself is conducted . Both are essential for developing effective machine learning models, but they serve different purposes and require different approaches for optimization and selection.

2.10 Model Evaluation Metrics

2.10.1 Accuracy, Precision, Recall, and F1-Score

The evaluation of machine learning models requires multiple metrics to capture different aspects of performance. The most fundamental metrics are derived from the confusion matrix, which contains True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN) .

- **Accuracy** is the ratio between correctly classified samples and the total number of samples, calculated as:
$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \in [0, 1] \tag{2.8}$$
However, accuracy can be misleading in cases of class imbalance.
- **Recall**, also known as sensitivity or True Positive Rate (TPR), denotes the rate of positive samples correctly classified:
$$\text{Recall} = \frac{TP}{TP + FN} \in [0, 1] \tag{2.9}$$
This metric is particularly important in medical applications (and cybersecurity) where missing positive instances should be minimized.
- **Precision** represents the proportion of correctly classified positive samples among all samples assigned to the positive class:
$$\text{Precision} = \frac{TP}{TP + FP} \in [0, 1] \tag{2.10}$$
High precision indicates that the model makes few false positive errors.
- **The F1-score** is the harmonic mean of precision and recall, providing a balanced measure:
$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \in [0, 1] \tag{2.11}$$
This metric is particularly useful when there is an uneven class distribution.

Confusion Matrix

A confusion matrix is defined as a table used to visualize the performance of a classifier, with rows and columns representing each class. For binary classification, it takes the form of a 2×2 matrix containing the counts of TP, TN, FP, and FN observations. The confusion matrix provides a comprehensive view of model performance, showing not only correct classifications but also the types of errors being made. This detailed breakdown is essential for understanding model behavior and identifying specific areas for improvement .

Receiver Operating Characteristic (ROC) Curve and Area Under Curve (AUC)

The Receiver Operating Characteristic (ROC) curve is a graphical plot that illustrates the diagnostic ability of a binary classifier as its discrimination threshold is varied. The curve plots the true positive rate (sensitivity) against the false positive rate ($1 -$ specificity) at various threshold settings. The Area Under the ROC Curve (AUC) provides a single scalar value that summarizes the performance across all possible thresholds .

AUC exhibits several desirable properties compared to overall accuracy: increased sensitivity in Analysis of Variance (ANOVA) tests, a standard error that decreases as both AUC and the number of test samples increase, decision threshold independence, and invariance to a priori class probabilities . For these reasons, AUC is often preferred over overall accuracy for single-number evaluation of machine learning algorithms.

Precision–Recall Curve

Precision-Recall (PR) curves are particularly useful for evaluating classifiers on highly skewed datasets. Unlike ROC curves, PR curves focus on the performance of the positive class, making them more informative when the negative class dominates the dataset. The curve plots precision against recall at various threshold settings, providing a clear picture of the trade-off between these two metrics .

A deep connection exists between ROC space and PR space, such that a curve dominates in ROC space if and only if it dominates in PR space However, there are important differences: in PR space, it is incorrect to linearly interpolate between points, and algorithms that optimize the area under the ROC curve are not guaranteed to optimize the area under the PR curve. This makes PR curves essential for evaluating models on imbalanced datasets common in intrusion detection tasks.

Training and Validation Loss Curves

Training and validation loss curves are fundamental tools for monitoring the learning process of neural networks and detecting issues such as overfitting and underfitting. During training, the optimization algorithm tries to minimize the cost function of the network, and the loss curves show how this metric changes over epochs .

Analysis of these curves allows for early detection of training issues, enabling timely interventions such as early stopping, learning rate adjustment, or model architecture modifications to improve performance and prevent wasted computational resources.

Part III

State of the Art

2.11 Introduction

This chapter provides a comprehensive review of recent literature addressing Network Intrusion Detection Systems (NIDS) utilizing advanced machine learning techniques. The purpose is to present the main datasets and methodologies that have been proposed.

To track the evolution of the field and highlight the gaps that necessitate our proposed architecture, the state-of-the-art papers are grouped thematically by their core approach to processing network data.

We transition from models that attempt to map network data into 2D visual representations, to those treating flows as NLP-style sequences, moving to cutting-edge models exploring Graph and Spatio-Temporal topologies, and finally the detection of zero-day attacks.

2.12 Datasets

To accurately simulate network environments, exercise testing and integrate proposed solutions, the cybersecurity community relies on baseline traffic captures. In this section, we present an overview of the most prominent benchmark datasets utilized in the state-of-the-art literature, highlighting their specific network topologies.

KDD-99:

The 1999 KDD intrusion detection contest uses a standard set of data [35] which includes a wide variety of intrusions simulated in a military network environment.The purpose is to audit and evaluate research in intrusion detection , Labs set up an environment to acquire nine weeks of raw TCP dump data for a local-area network (LAN) simulating a typical U.S. Air Force LAN. .

The raw training data was about four gigabytes of compressed binary TCP dump data from seven weeks of network traffic. This was processed into about five million connection records. Similarly, the two weeks of test data yielded around two million connection records.

Attacks fall into four main categories

- **DOS (Denial of Service):** Attempts to shut down or overload a network (e.g., SYN flood).
- **R2L (Remote to Local):** Unauthorized remote access attempts (e.g., password guessing).
- **U2R (User to Root):** Unauthorized escalation of local privileges (e.g., buffer overflows).
- **Probing:** Surveillance and scanning to find vulnerabilities (e.g., port scanning).

NSL-KDD:

NSL-KDD [36] is a dataset suggested to solve some of the inherent problems of the KDD'99 dataset

CIC-IDS2017:

CIC-IDS2017 [37], was developed by the Canadian Institute for Cybersecurity (CIC), represents a critical milestone in NIDS evaluation. It was specifically engineered to address the severe architectural shortcomings and lack of modern threat representation found in legacy benchmarks like KDD Cup 99 and NSL-KDD [1].

Unlike older, purely synthetic datasets, **CIC-IDS2017** captures highly realistic background traffic which extracts and simulates the abstract behavior of genuine human interactions (e.g., HTTP, HTTPS, FTP, SSH, and email protocols). The dataset encompasses five days of dense network activity, featuring a comprehensive taxonomy of modern attack vectors categorized as follows:

- *DDoS* and *Dos*
- *Web Attacks* and *Brute Force*
- *Infiltration* and *PortScans*
- *Botnets*

CIC-IDS2018:

The CSE-CIC-IDS2018 [38] dataset is a large-scale, dynamically generated benchmark. Created as a collaborative project between the Communications Security Establishment (CSE) and the Canadian Institute for Cybersecurity (CIC) on the AWS cloud computing platform, this dataset addresses the critical limitations of older static benchmarks by providing realistic, up-to-date traffic compositions. It is a direct evolution of the CIC-IDS2017 dataset.

To synthesize a highly realistic network environment, the dataset utilizes a profile-based generation methodology. The key characteristics of this approach include:

- **B-Profiles (Benign Profiles):** These encapsulate the naturalistic behavior of human users by leveraging machine learning and statistical analysis techniques to simulate standard protocols such as HTTP, HTTPS, FTP, SSH, and email.
- **M-Profiles (Malicious Profiles):** These introduce seven distinct, modern attack scenarios into the network, specifically: Brute-force, Heartbleed, Botnets (Zeus and Ares), DoS, DDoS, Web attacks, and internal network infiltration.
- **Diverse Topology:** The dataset simulates a complex organizational infrastructure comprising 5 departments, 30 servers, and 420 victim machines running various Windows and Linux operating systems. This internal network is targeted by an external attacking infrastructure of 50 machines.
- **Comprehensive Data Sources:** For advanced cybersecurity and machine learning research, the dataset is exceptionally thorough. It provides raw network traffic captures (PCAPs) and system event logs (from both Windows and Ubuntu environments) for researchers who wish to build custom feature extractors.

UNSW-NB15:

The **UNSW-NB15** dataset is a comprehensive network intrusion detection benchmark generated in the Cyber Range Lab of UNSW Canberra [39]. Created to provide a highly realistic evaluation environment, the dataset utilizes the *IXIA PerfectStorm* emulation tool to synthesize a hybrid manifold of real, modern, normal network activities alongside contemporary synthetic attack behaviors.

The underlying composition of the dataset is organized as follows:

- **Traffic Volume and Ingestion:** The raw network captures consist of approximately 100 GB of raw traffic captured via the `tcpdump` utility and archived in native PCAP format.
- **Attack Taxonomy:** The malicious traffic profiles encompass nine distinct structural categories of cyber-attacks:
  - *Fuzzers* and *Analysis*
  - *Backdoors* and *Exploits*
  - *DoS* and *Worms*

– *Generic*, *Reconnaissance*, and *Shellcode*

- **Feature Extraction Pipeline:** To facilitate advanced machine learning applications, the dataset creators deployed a pipeline leveraging `Argus`, `Bro-IDS` (Zeek), and twelve specialized custom heuristic algorithms to extract 49 distinct multivariate features embedded with corresponding ground-truth class labels.

In its entirety, the complete dataset contains $2,540,044$ connection records structurally distributed across four tabular CSV files, which are accompanied by a dedicated ground-truth table and an event list.

To guarantee reproducibility and uniform baseline testing across comparative research studies, a standardized subset has been formally partitioned into two non-overlapping sets: a dedicated training set containing $175,341$ records and a test set containing $82,332$ records. Both partitions maintain a complex, mathematically balanced mixture of benign baseline communication profiles and active threat vectors.

2.13 Database normalization

Database normalization is a fundamental database design principle utilized to structure data in a consistent and organized manner. The primary purpose of normalization is to reduce complexity, eliminate data duplication, and avoid data redundancy. By dividing data into multiple tables linked through relationships—established via primary keys, foreign keys, and composite keys—normalization ensures the integrity of the database. Furthermore, it is critical for eliminating undesirable characteristics and anomalies associated with the insertion, deletion, and updating of data records.

Normalization is implemented through a series of progressive stages, known as normal forms. These forms are cumulative, meaning that each subsequent level builds upon the foundational rules of the levels beneath it. While normalization can theoretically extend to the fourth, fifth, and even sixth normal forms depending on data size and system requirements, the Third Normal Form (3NF) is the most commonly achieved standard in standard relational database design. [40]

First Normal Form (1NF)

For a table to be in the first normal form, it must meet specific criteria. A single cell must not hold more than one value, which establishes atomicity . Furthermore, there must be a primary key for identification, there can be no duplicated rows or columns, and each column must have only one value for each row in the table .

Second Normal Form (2NF)

Because 1NF only eliminates repeating groups and not redundancy, the second normal form is required. A table is considered to be in 2NF if it is already in 1NF and possesses no partial dependency . This means that all non-key attributes are fully dependent on a primary key . For example, if a table has a composite primary key but certain columns only depend on one part of that key, those columns must be separated into a different table to achieve 2NF .

Third Normal Form (3NF)

Even when a table is in 2NF and eliminates repeating groups and redundancy, it does not eliminate transitive partial dependency . Transitive partial dependency occurs when a non-prime attribute

(an attribute that is not part of the candidate's key) is dependent on another non-prime attribute . Therefore, for a table to achieve 3NF, it must be in 2NF and have no transitive partial dependency . While further normal forms like 4NF and 5NF exist depending on data size and requirements, 3NF is the most common normal form used .

2.14 Related Research

In this section, we review the existing research and publications that are related to our work. Examining these papers allows us to establish a clear comparative baseline and contextualizes our approach within the current state of the art.

2.15 Visual Representation and Image-Based Models

A significant branch of recent research attempts to bypass the limitations of tabular data by mathematically transforming network packets and flows into 2D. This allows the utilization of mature Computer Vision architectures (like CNNs and ViTs) for anomaly detection.

2.15.1 Xu & al., "Falcon" 2021

Introduces [41], an Android malware detection framework that treats network traffic classification as a 2D image sequence classification problem.The framework was trained and evaluated using the **Android Malware CICMal2017 dataset**, which includes 426 malware and 1700 benign samples. From 2,126 raw PCAP files, the authors extracted over 3,255,391 total network flows.

Architecture and Solution

- **Network Flows to Images:** Packets are converted into 2D grayscale images by trimming or padding flows to ensure a uniform 784-byte size.
- **Continuous Traffic Processing:** A pre-trained 8-layer CNN extracts vectors, which are then passed through a bi-directional LSTM network to capture temporal sequential relationships between the 2D packet images.

Experimental Results

- Evaluated on the used Dataset achieved an accuracy of 97.16% in binary classification and 84.70% in multi-class categorization. The model was tested against various classifiers, including AdaBoost, GradientBoost, MLP, and DecisionTree. The **Random Forest** classifier achieved the best performance with an accuracy of 97.16, precision of, 97.13, recall of 97.16, and an F1-score of 97.09.

2.15.2 L. Mohammadpour & al., "CNN-Based IDS Survey" 2022

This survey [42] provides a comprehensive study on a CNN-based IDS, categorizing it into four main approaches: single CNN models, hybrid CNN-RNN architectures (combining CNNs with LSTMs or GRUs for spatial-temporal feature extraction), hybrid CNNs with deep learning models (like autoencoders), and hybrid CNNs paired with machine learning algorithms.It also analyzes architectures, hyperparameter settings, and input dimension types (1D vs. 2D).

Architecture and Solution

- Highlights preprocessing methods used to convert tabular network packets into 2D formats (grayscale matrices) to extract hierarchical correlations without manual feature engineering.

Experimental Results

- Reviews KDD Cup99, NSL-KDD, UNSW-NB15, and CIC-IDS2017. Confirms that hybrid CNN models (e.g., CNN-LSTM) consistently outperform single CNNs by capturing temporal sequences alongside spatial properties.

2.15.3 Kim & al., "Multiple Image Transformers" 2023

[43] introduces an optimized method for converting tabular NIDS datasets into 2D visual representations utilizing multiple image transformers, integrating dimensionality reduction to construct RGB images.

Architecture and Solution

- **RGB Image Synthesis:** Employs algorithms (t-SNE, UMAP, PCA) to map numerical features into 2D spatial matrices. The top three are mapped to Red, Green, and Blue channels to synthesize a multi-dimensional RGB image.
- **Dual CNN Classifier:** Utilizes a four-layer dual CNN with parallel convolutional filters to process the generated RGB images.

Experimental Results

- This survey proposed multiple-transformer ensemble using 3-channel color images achieved superior performance with an **Accuracy** of **95.60** and an **F1-Score** of **93.86** on the ISCX-IDS2012 dataset. On the CSE-CIC-IDS2018 dataset, the method achieved an **Accuracy** of **95.50** and an **F1-Score** of **92.52**

2.15.4 He & al., "DE-VIT" 2024

This study [44] introduces a Deformable Vision Transformer (DE-VIT) to address network intrusion detection by transforming data into image formats. It leverages deformable attention mechanisms to capture global and local features while minimizing computational costs.

Architecture and Solution

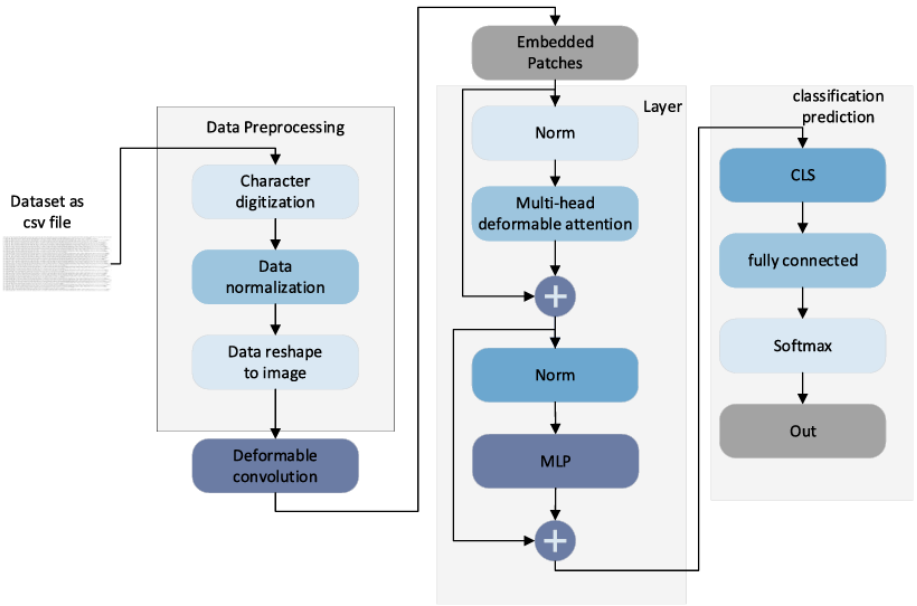

Figure 2.5: Model process [44]

- **Data Transformation:** Preprocesses traffic into 64 features and reshapes them into an 8×8 image matrix. It applies deformable convolutions with a sliding window to extract image features.
- **Deformable Attention:** Instead of dense self-attention, the model calculates relationships only with surrounding features, reducing overhead.
- **L-Focal Loss:** Implemented to address highly imbalanced datasets.

Experimental Results

- Achieved an **Accuracy** of 99.5% on CIC-IDS2017 [45] and 97.25% on UNSW-NB15[45] and produced an exceptional Area Under the Curve **(AUC)** of 0.995 in binary configurations, significantly outperforming CNN and LSTM baselines.

2.15.5 Abed & al., "A modified CNN-IDS model" 2024

This study [46] addresses supersized data spaces by implementing feature selection techniques prior to image-based CNN classification.

Architecture and Solution

- **Dimensionality Reduction:** Employs PCA and Singular Value Decomposition (SVD) to mathematically transform the feature space and discard irrelevant noise.
- **Hybrid Classification:** Evaluates the transformed space using Ridge Regression, Stochastic Gradient Descent, and CNNs to execute threat detection.

Experimental Results

- The study achieved a successful implementation with a **Precision** around 86.93 to 88.52 precision for UNSW-NB15, and about 93.81 to 99.67 for the WSN-DS dataset.

2.15.6 Xiao Liu & al., "Mean Masked Autoencoder with Flow-Mixing for Encrypted Traffic Classification" 2026

Recently, the application of Masked Autoencoders (MAE) for encrypted traffic classification has highlighted the limitations of standard random masking, which is ill-suited for the high semantic density of network packets. To overcome this constraint, Liu et al. (2026) proposed the Mean Masked Autoencoder (MMAE) [47], an architecture leveraging a teacher-student self-distillation paradigm coupled with a Flow-Mixing strategy. This approach replaces conventional masking by injecting cross-flow interference, thereby compelling the model to discriminate features within highly noisy environments.

While this method improves model robustness, it remains restricted to a strictly spatial modeling of the traffic. In contrast to this flow-mixing approach, our architecture resolves temporal feature extraction through direct modeling in the spectral domain. By introducing Masked Frequency Modeling (MFM) via Fast Fourier Transform (FFT), our foundation model captures the intrinsic spatio-temporal dynamics of an isolated flow, thus bypassing the computational complexity associated with asymmetric flow pairing.

2.16 Sequential Flow and NLP-Adapted Transformers

While visual models successfully mapped spatial correlations, they often struggle with long-term temporal dependencies. Consequently, another major research branch treats network flows identically to sentences in Natural Language Processing (NLP), applying standard sequence-based Transformers.

2.16.1 Athul K & Anita John., "A Deep Learning based Intrusion Detection System using Transformers" 2022

This study [48] focuses on minimizing computational complexity by ranking and selecting important security features using NLP models before classification.

Architecture and Solution

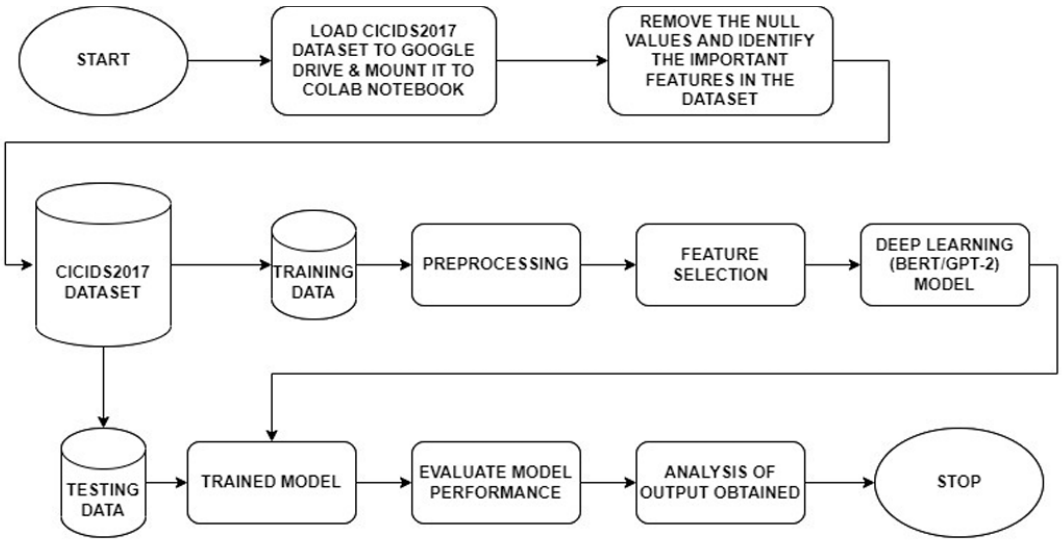

Fig. 1. System Architecture

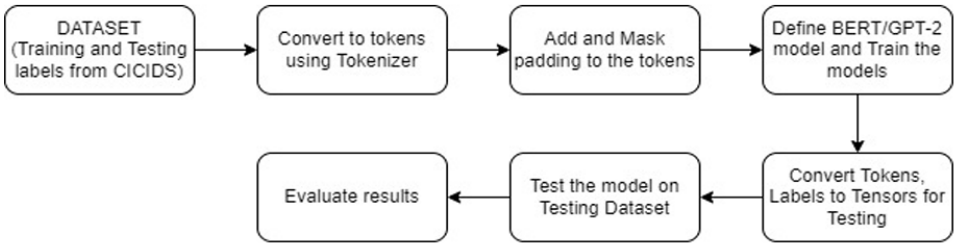

Figure 2.6: System Design [48]

- **Feature Selection Classification:** The dataset is preprocessed to identify and utilize only the 30 most important features to reduce computational complexity and overhead.
- **Data Transformation Classification:** The framework converts the intrusion and normal records into strings of text sequences. These sequences are then tokenized and parsed into chunks of length six.
- **Padding and Masking:** The tokens are appended with padding and masked, which is a necessary step for the transformer architectures to function properly.
- **Classification:** The processed data is fed into the defined BERT and GPT-2 models for binary classification

Experimental Results

- Evaluated on CICIDS2017. Proved that leveraging NLP transformers for feature dimensionality reduction minimizes computational complexity while maintaining high detection rates.

2.16.2 Wu & al., "RTDIS" 2022

[49] serves as a foundational baseline demonstrating how transformers effectively handle high-dimensional traffic data without losing critical feature representations.

Architecture and Solution

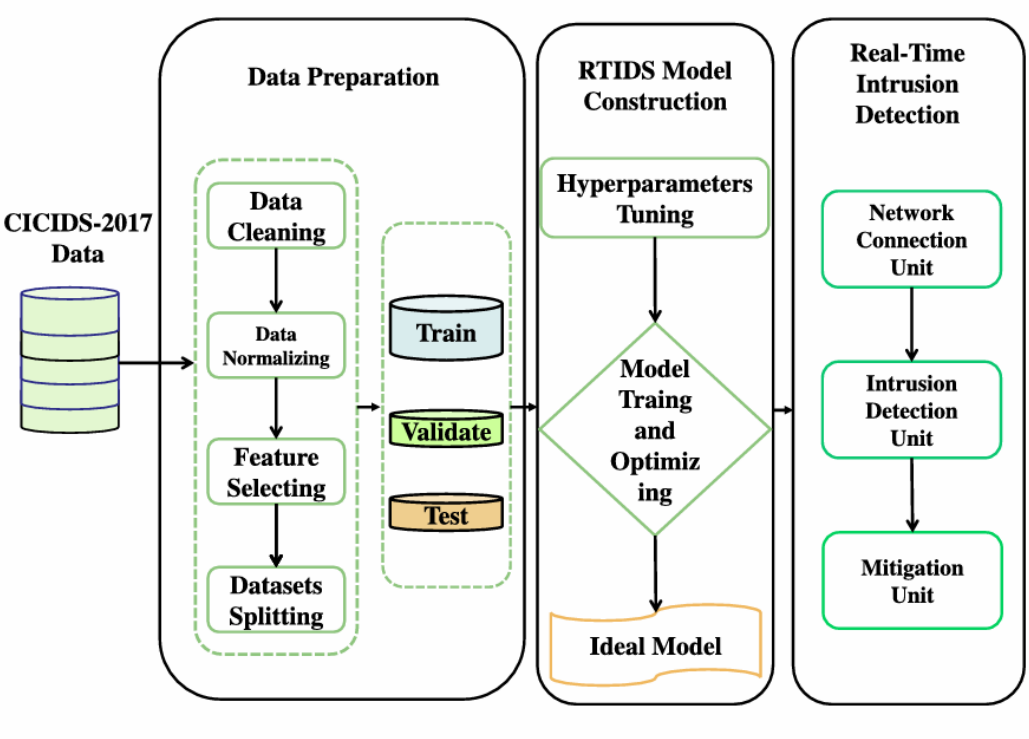

Figure 2.7: RTDIS model architecture [49]

- **Data Balancing:** To combat the severe class imbalance in the datasets and prevent model overfitting, the framework utilizes the Synthetic Minority Oversampling Technique (SMOTE) during preprocessing.
- **Self-Attention Mechanism:** A multi-head self-attention mechanism is applied to learn varying weights of feature representations, bypassing traditional recurrent connections.
- **Masked Decoders for Robustness:** The decoder stack includes an additional masked self-attention sublayer that randomly masks a portion of features.

Experimental Results

- RTIDS achieved an F1-Score of 99.17% on CICIDS2017 and 98.48% on CIC-DDoS2019, outperforming baseline models including SVM, RNN, and LSTM.

2.16.3 Liu & al., "Intrusion Detection Model Based on Improved Transformer" 2023

[50] addresses three challenges: lengthy training times, inaccurate detection of overlapping classes, and poor multi-class classification by optimizing positional encoding for non-sequential data.

Architecture and Solution

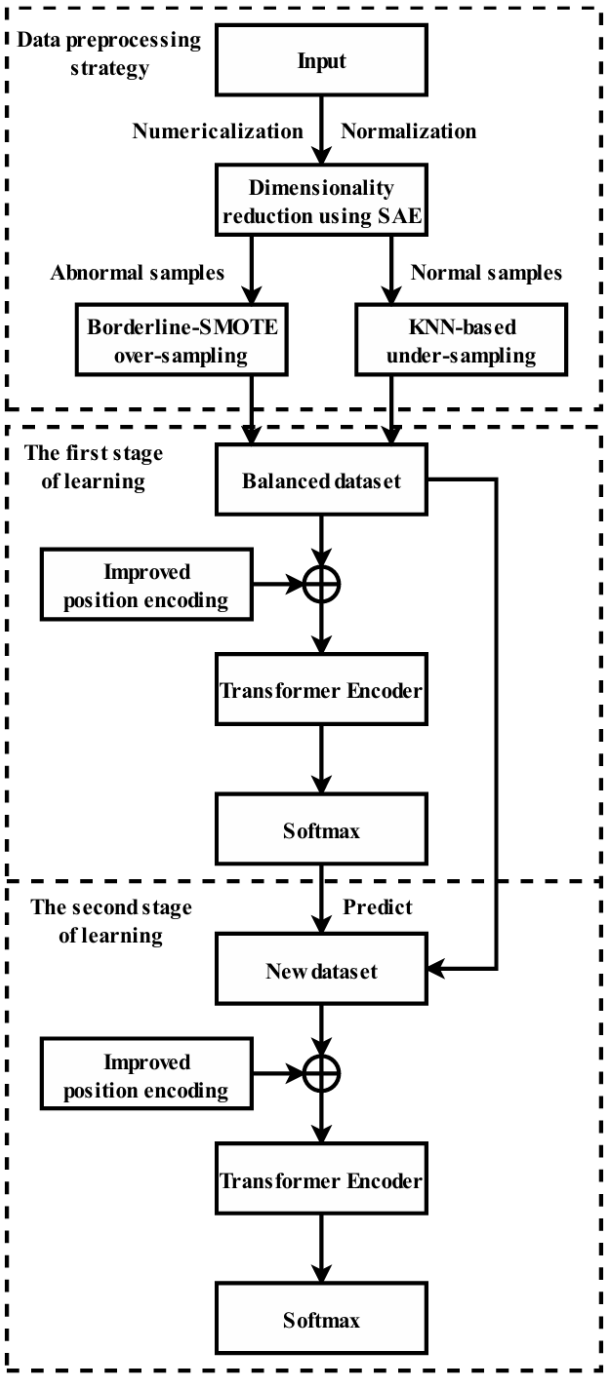

Figure 2.8: Improved Transformer-based intrusion detection model [50]

- **Improved Position Encoding:** Standard Transformer encoding is ineffective for independent numerical features. The authors modified the formula to embed the positional index of each feature (i) to learn spatial dependencies.
- **Hybrid Sampling & Two-Stage Learning:** Uses KNN under-sampling and Borderline-SMOTE, followed by a rough binary classification (normal vs. attack) leading into an accurate multi-class prediction.

Experimental Results

- On the NSL-KDD dataset, the model achieved an accuracy of 88.7% and an F1-score of 88.2% for binary classification, with a remarkably fast training time of only 8 seconds. For multi-class classification, it recorded an 84.1% accuracy and an 83.8% F1-score. On the UNSW-NB15 dataset, the model achieved 87.5% accuracy and an 87.3% F1-score for multi-class classification, completing training in just 19 seconds.

2.16.4 Manocchio & al., "FlowTransformer" 2024

This study [51] evaluates various transformer components (input encoding, blocks, heads) for flow-based network traffic. It aims to identify the optimal configurations for NIDS by comparing various transformer architectures, input encodings, and classification heads.

Architecture and Solution

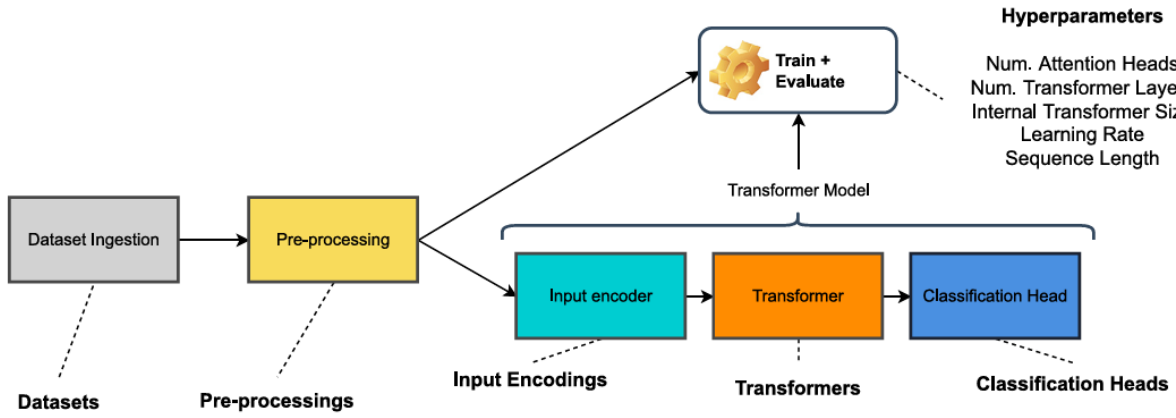

Figure 2.9: FlowTransformer framework architecture [51].

- **Shallow vs. Deep Transformers:** Reveals that shallow transformers (e.g, 2 layers) perform equivalently to deep models (BERT/GPT) in flow-based tasks while offering vastly superior computational throughput.
- **Classification Head:** Proves that the "Last Token" approach significantly outperforms Global Average Pooling for NIDS contexts.

Experimental Results

- It achieved a state-of-the-art F1-Score of 98.00% on NSL-KDD, 97.05% on CSE-CIC-IDS2018, and 90.45% on UNSW-NB15.

2.16.5 F.Ullah & al., "IDS-INT" 2024

In this study [52] the authors created a deep learning-based Intrusion Detection System (IDS) that applies Natural Language Processing (NLP) algorithms—specifically BERT and GPT-2 to detect modern cyber threats.

Architecture and Solution

- **Semantic Tokenization:** Parses PCAP files to extract flow events. It utilizes a BERT large model to compute contextual relationships and extract semantic train features.
- **CNN-LSTM Classifier:** Feeds the BERT-extracted features into a 1D-CNN and LSTM network to capture sequential and temporal dependencies.

Experimental Results

- Tested on UNSW-NB15, CIC-IDS2017, and NSL-KDD. It successfully mitigated majority-class bias and significantly outperformed the baseline methods with an overall 99% F1-score, and 99.21% Accuracy.

2.16.6 Berry & al., "Explainability of Network Intrusion Detection Using Transformers" 2025

This study [53] shifts focus from traditional flow-based analysis to a packet-level approach. It addresses open-world recognition by leveraging transformers to generate natural language explanations for unknown network packets.

Architecture and Solution

- **Packet Embedding & Clustering:** Utilizes a fine-tuned **Bert** model to generate 768-dimensional contextual embeddings for individual packets based on the first 500 payload bytes. It applies k-means clustering to assign network tags.
- **Explainability:** A Falcon-7B decoder-only model is fine-tuned to generate human-readable descriptive sentences explaining the assigned tags and payload contents.

Experimental Results

- Evaluated on CIC-IDS2017 and UNSW-NB15. The fine-tuned BERT model achieved an accuracy of 99.48 and a macro F1-score of 98.46% on a test set containing over 500,000 packets. Also achieved an explainability score of 0.6331 and generated text with 95.53% accuracy for packet details at a speed of 2 seconds per packet.

2.17 Graph and Topological Network Modeling

Recognizing the network traffic is neither purely a flat 2D image nor a simple 1D sentence, the most recent frontier of NIDS research addresses the structural topology of networks. These models attempt to map the complex interconnections (spatial graphs) of network nodes and flows.

2.17.1 Caville & al., "Anomal-E" 2023

[54] investigates the application of Graph Neural Networks (GNNs) for self-supervised network anomaly detection for unseen zero day attack, capturing lateral movement paths found in APTs without labeled traffic.

Architecture and Solution

- **Edge-leveraging GNN (E-GraphSAGE):** Instead of standard node-based GNNs, the framework uses an E-GraphSAGE encoder to capture flow information and topological patterns.
- **Self-Supervised Learning (Modified DGI):** It applies a modified Deep Graph Infomax (DGI) methodology. It generates a "corrupted" or negative graph by randomly shuffling the edge features while keeping the graph structure intact. The model is trained to maximize the mutual information between the extracted edge embeddings and the global graph summary.
- **Downstream Anomaly Detection:** Once the E-GraphSAGE encoder extracts the latent edge embeddings, these are fed into traditional unsupervised novelty/anomaly detectors to do the binary classification. The authors tested four algorithms: PCA, Isolation Forest (IF), Clustering-Based Local Outlier Factor (CBLOF), and Histogram-Based Outlier Score (HBOS).

Experimental Results

- The Anomal-E-HBOS combination achieved the highest average Macro F1-score of 91.48% across the datasets. The Anomal-E-CBLOF configuration achieved an average Macro F1-score of 93.04%, followed closely by Anomal-E-HBOS at 91.89%.

2.17.2 Appiahene & al., "NIDS Using a Hybrid Graph-Based and Transformer Architecture" 2026

[55] Presents **GConvTrans (Graph Convolution Transformer Network)** a hybrid model combining a graph convolutional layer and a transformer encoder to address limitations in capturing both localized topological connections and global context within network data in distributed cloud environments.

Architecture and Solution

- **Graph Convolution Layer:** Transforms tabular data into computational graphs. Normalizes the adjacency matrix and aggregates information from immediate neighbors to encode local structural patterns.
- **Transformer Encoder Layer:** Refined embeddings are processed through a Transformer utilizing multi-head self-attention to capture long-range dependencies and global context from distant nodes.

Experimental Results

- Evaluated on the CSE-CIC-IDS2018 dataset (DDoS subset). Achieved a testing accuracy of 96.94%. Proves that combining graph convolutions and transformer encoders is highly efficient for capturing sophisticated patterns.

2.18 Zero-Day Attack Detection

2.18.1 Hindy & al., "Utilising Deep Learning Techniques for Effective Zero-Day Attack Detection" 2020

This paper [56] aims to propose a lightweight Intrusion Detection System (IDS) based on Deep Learning to effectively flag zero-day attacks.The authors focus on building a model that achieves a

high recall for detecting unknown attacks while minimizing the false negatives.

Approach Used

The researchers utilize an **Autoencoder** designed to act as a self-supervised outlier detector.

- **Training on Benign Traffic:** The autoencoder is trained exclusively on normal/benign network traffic (using a 75% training and 25% validation split) so that it accurately learns the patterns of normal and usual behavior. Hence the simulation of the zero-day attack was done on the Dos and DDos detection.
- **Zero-Day Detection:** During the evaluation phase, an attack is fed into the system. If the **Mean Squared Error (MSE)** which measures the reconstruction error between the original network traffic instance and the instance decoded by the autoencoder is larger than a predefined threshold, the traffic is flagged as an anomalous zero-day attack.
- **Baseline Comparison:** To prove the model's efficacy, its performance is compared against a **One-Class Support Vector Machine (SVM)**, which is a well-established unsupervised outlier detection model.

Experimental Results

The findings reveal that the autoencoder is highly effective at detecting complex zero-day attacks and significantly outperforms the One-Class SVM baseline.

- **CICIDS2017 Metrics:** The autoencoder achieves a global detection accuracy of 75% to 98%. The detection accuracy peaks at 90.01% for DoS GoldenEye, 98.43% for DoS Hulk, 98.47% for Port scanning, and 99.67% for DDoS attacks. The accuracy on benign traffic (specificity) relies on the chosen MSE threshold, achieving 95.19%, 90.47%, and 81.13% for thresholds of 0.15, 0.1, and 0.05, respectively.
- **NSL-KDD Metrics:** The overall zero-day detection accuracy ranges between 89% and 99%. More precisely, the model attains overall accuracies of 91.84%, 92.96%, and 91.84% using thresholds of 0.3, 0.25, and 0.2, respectively.
- **On the specific KDD test+ evaluation dataset**, the autoencoder correctly detects DoS at 94.67%, R2L at 95.95%, and U2R at 83.78%.

2.18.2 Sarhan & al., "From Zero-Shot Machine Learning to Zero-Day Attack Detection" 2021

This paper [57] introduces a Zero-Shot Learning (ZSL) methodology to evaluate and improve how well ML models can generalize to detect completely unseen attacks in a post-deployment scenario.

Used Approach

The researchers apply Zero-Shot Learning (ZSL) using Multi-Layer Perceptron (MLP) and Random Forest (RF) classifiers. During the training phase, one specific attack class is intentionally excluded to simulate a zero-day scenario. The model learns the distinctive semantic attributes of malicious behavior from the known attacks. During the inference phase, the model uses these learned attributes to detect the unseen attack class. The Wasserstein Distance (WD) is also used to analyze the statistical differences between the feature distributions of known and unknown attacks.

Experimental Results

Evaluation was done on **UNSW-NB15** and **NF-UNSW-NB15-v2** (a more recent NetFlow-based version of the UNSW-NB15). On one hand the MLP classifier achieves an average Zero-day Detection Rate (Z-DR) of 85.5%, while the RF averages 80.67%. Both models easily detect attacks like Generic, DoS, and Worms (Z-DR $>$ 90%), but fail significantly on Fuzzers (around 15-20%). On the other hand performance improves with the NetFlow version-2 features; MLP reaches a Z-DR of 92.45% and RF reaches 87.37%.

2.18.3 Kumar & al., "A robust intelligent zero-day cyber-attack detection technique" 2021

[58] proposes an intelligent model combining "heavy-hitters" and graph-based techniques to detect both High Volume Attacks (HVA) and Low Volume Attacks (LVA) without relying on known signatures.

Datasets

Evaluation was done on two types of data:

- **Real-Time Data:** Traffic generated in a controlled virtual environment containing normal traffic alongside DoS/DDoS (HVA) and Probe/Data Theft/Keylogging (LVA) attacks.
- **CICIDS18** Benchmark

Used Approach

The model processes raw network traffic as hexadecimal byte streams and operates in two main modules:

- **HVA Module:** Uses a heavy-hitter algorithm to extract frequent k-gram tokens from the byte stream. Tokens that frequently appear in attack traffic but rarely in normal traffic (exceeding a threshold) form HVA signatures.
- **LVA Module:** Tokens that fail the HVA threshold are passed to the LVA module. A directed graph is constructed where vertices are tokens and edges are their consecutive occurrences. A scoring function assigns weights to vertices, and paths exceeding a certain threshold are consolidated into zero-day LVA signatures.

Experimental results

- **Real-Time Data:** For binary classification, the model achieves an Accuracy of 91.33%, Precision of 99.77%, Recall of 89%, F-Measure of 94.1%. For multi-class classification, it achieves 90.35% Accuracy and an Attack Detection Rate of 87.5%.
- **CICIDS18 Data:** It reaches an Accuracy of 91.62%, Precision of 99.99%, Recall of 91.48%, and an F-Measure of 95.54%.

2.18.4 Hairab & al., "Anomaly Detection of Zero-Day Attacks Based on CNN and Regularization Techniques" 2023

[59] evaluates how the integration of regularization techniques (L1 and L2) into CNN can mitigate overfitting and substantially improve the detection of zero-day IoT attacks.

Dataset

The models are trained and tested using the modern **TON-IoT** dataset (network variant). The training dataset is composed exclusively of normal traffic and DDoS attacks.

Used Approach

Nine relevant network features (such as protocol, duration, and bytes sent/received) are extracted and encoded.

These features are shaped into 3×3 matrices to simulate single-channel grayscale images. A standardized CNN architecture is employed in three variants: a standard (non-regularized) CNN, a CNN with L1 regularization (factor 0.005), and a CNN with L2 regularization. Classical ML classifiers (Logistic Regression, Naive Bayes, AdaBoost) are used as baselines.

Experimental results

The testing was done onto two different scenario attacks:

- **Scenario A (Backdoor):** Regularized CNNs vastly outperform other models. The CNN L2 model achieves 97.94% Accuracy, 98.31% Recall, 98.27% F1 Score, and an AUC of 0.98. The non-regularized CNN only manages 83.15% Accuracy, while the best classical ML model (Naive Bayes) achieves just 61.31%.
- **Scenario B (Scanning):** The CNN L1 model performs best, attaining 87.59% Accuracy, 96.81% Recall, 88.81% F1 Score, and an AUC of 0.8899. The standard CNN hits 77.52% Accuracy, while AdaBoost falls to 31.69%.

2.18.5 Appani & al., "Self-Supervised Representation Learning for Zero-Day Attack Detection in Encrypted Network Traffic" 2023

[60] proposes a Self-Supervised Learning (SSL) framework to extract discriminative feature representations directly from encrypted traffic without needing labeled training data.

Used Approach

The architecture is a hybrid of Transformer encoders and Temporal Convolutional Networks (TCNs). Because the payloads are encrypted, the model relies on self-supervised "pretext tasks" to learn underlying traffic patterns. These tasks include:

- **Flow Reconstruction:** Masking 15-20% of packet headers and forcing the model to recover them.
- **Contrastive Learning:** Using NT-Xent loss to maximize the similarity between augmented versions of the same flow while distancing it from dissimilar flows.

Anomaly scoring is computed using the Mahalanobis distance in the latent space; flows distant from the centroid of benign training data are flagged as zero-day attacks.

Experimental results

The models are evaluated using two large-scale public datasets containing encrypted traffic: **CIC-IDS2017** and **UNSW-NB15**. The research concluded to these results:

- **Zero-Day Detection:** The SSL model achieves an F1-Score of 88%, a False Positive Rate of 1.80%, and an AUC of 0.98 on unseen attacks. This represents a 22% improvement in the F1-Score compared to standard Autoencoders, and a 15% reduction in FPR compared to Isolation Forests.
- **Cross-Dataset Generalizability:** Training on CIC-IDS2017 and testing on UNSW-NB15 yields a strong F1-Score of 85.3%, proving the embeddings do not overfit to specific network topologies.

2.18.6 Armijos & al., "Zero-day attacks: review of the methods used based on intrusion detection and prevention systems" 2024

[61] reviews zero-day attack mechanics and surveys the methodologies deployed by modern Intrusion Detection Systems (IDS) and Intrusion Prevention Systems (IPS), specifically comparing traditional approaches against contemporary Machine Learning and Deep Learning strategies.

Used Approach

The study provides a comparative theoretical analysis distinguishing between:

- **SIDS (Signature-Based IDS):** Relies on matching traffic to known rule sets (e.g., Snort).
- **AIDS (Anomaly-Based IDS):** Leverages ML algorithms (e.g., SVM, Decision Trees, Naive Bayes) and DL architectures (e.g., Neural Networks, Autoencoders) to learn normal network baselines and detect deviations representing zero-day threats.

Experimental results

The review evaluates many datasets, including: NSL-KDD, UNSW-NB15, CICIDS2017, CIC-AWS-2018.

- **SIDS Inefficiency:** Signature-based tools like Snort only manage to detect an estimated 8.2% of zero-day attacks.
- **AIDS/ML Performance:** Semi-supervised ML techniques, such as combining One-Class SVM with Benford's law, achieve an F1-Score of 86%. Other classic classifiers like SVM and Decision Trees reach around 95.1% Accuracy.
- **DL Superiority:** Neural Networks and Autoencoders vastly outperform other methods. Autoencoders evaluated on NSL-KDD yield Accuracies between 89% and 99%, and between 75% and 98% on CICIDS2017. The review confirms DL as the most capable approach for modeling zero-day evasions.

2.18.7 Li & al., "SAFE" 2025

Because machine learning approaches suffer from zero-day detection flaws due to labeled data dependency, the authors propose [62] this framework that learns representations of network behavior , to detect any novel behavior.

Used Approach

First, SAFE restructures conventional tabular network intrusion data into 2D image-like matrices by leveraging Principal Component Analysis (PCA) to rank and select features. The core relies on a **Masked Autoencoder** that learns comprehensive behavioral representations from the unlabeled network mapping.

Finally, the latent spatial features extracted by the encoder head of the MAE are paired with a lightweight anomaly/novelty detector (e.g., Local Outlier Factor - LOF) during inference to recognize emerging cyber-attacks.

Experimental results

Evaluation was done across four extensive IIoT datasets: X-IIoTID, WUSTL-IIoT, Edge-IIoTset, and MQTTset.

[62] achieved an average F1-score of 96.79% across all datasets, specifically scoring 96.41% on X-IIoTID, 99.40% on WUSTL-IIoT, 99.95% on Edge-IIoTset, and 91.38% on MQTTset.

2.18.8 Nayak & al., "ThreatFormer-IDS" 2026

[63] proposes **ThreatFormer-IDS**, a unified Transformer-based sequence modeling framework that aims to reliably detect intrusions at low false-positive rates while maintaining performance against evolving traffic, generalizing to unseen attack families.s.

Datasets

The framework is benchmarked using the **ToN IoT** dataset, consisting of heterogeneous network traffic logs generated in a realistic Edge/Fog-Cloud testbed.

Used approach

The architecture converts network flow records into time-ordered sequences. It utilizes four core components:

- **Weighted Supervised Learning:** Handles class imbalances to ensure low False Positive Rates (FPR).
- **Masked Self-Supervised Learning (SSL):** Improves representation stability by forcing the Transformer to reconstruct masked parts of the sequence.
- **Adversarial Training (PGD):** Employs Projected Gradient Descent with scale-normalized perturbations to defend against feature-level evasion.
- **Explainable Attribution:** Uses Integrated Gradients (IG) to generate attribution maps explaining which time steps and features triggered an alert.

A strict zero-day evaluation protocol holds out specific attack families completely during training.

Experimental resuts

- **Chronological (In-Distribution) Test:** Achieves AUC-ROC of 0.994, AUC-PR of 0.956, FPR of 0.910, and F1 of 0.976, outperforming baselines like LightGBM, 1D-CNN, and BiLSTM.
- **Zero-Day Generalization:** On held-out attack families, the model maintains a superior AUC-PR of 0.721 and FPR of 0.783.

2.19 Synthesis and Comparative Analysis

To clarify and exhibit the evolution of IDS, this table 2.1 provides a comprehensive comparison of the state-of-the-art models discussed in this chapter.

Table 2.1: Comparative Summary of State-of-the-Art NIDS Architectures

Research Paper	**Release Date**	**Dataset Used**	**Result**
[56]	2020	CICIDS2017, NSL-KDD	**Acc:** 75% - 98% (CIC), 89% - 99% (NSL-KDD)
[41]	2021	CICMal2017	**Acc:** 97.16% (Binary), 88.32% (Multi-class)
[57]	2021	UNSW-NB15, NF-UNSW-NB15-v2	**Z-DR (MLP):** 85.5% to 92.45%
[58]	2021	Real-Time Data, CICIDS18	**Acc:** 91.33% - 91.62% **F-Measure:** 94.1% - 95.54%
[42]	2022	KDD Cup99, NSL-KDD, UNSW-NB15, CIC-IDS2017	Hybrid CNN-LSTM consistently outperforms single CNNs
[49]	2022	CICIDS2017, CIC-DDoS2019	**F1:** 99.17% (CIC), 98.48% (DDoS)
[48]	2022	CICIDS2017	Minimized computational complexity while maintaining high detection
[43]	2023	ISCXIDS2012, CSE-CIC-IDS2018	**Acc:** 95.50% - 95.60% **F1:** 92.52% - 93.86%
[50]	2023	NSL-KDD, UNSW-NB15	**Acc:** 87.5% - 88.7% **F1:** 87.3% - 88.2%
[54]	2023	Modern Benchmark Datasets	**Macro F1:** 91.48% - 93.04%
[59]	2023	TON-IoT	**Acc:** 97.94% **F1:** 98.27% (Backdoor)
[60]]	2023	CIC-IDS2017, UNSW-NB15	**F1:** 88% **FPR:** 1.80% **AUC:** 0.98
[44]]	2024	CIC IDS2017, UNSW-NB15	**Acc:** 99.5% (CIC), 97.25% (UNSW) **AUC:** 0.995
[46]]	2024	UNSW-NB15, WSN-DS	**Acc:** 86.93% - 88.52% (UNSW), 93.81% - 99.67% (WSN-DS)
[51]	2024	NSL-KDD, CSE-CIC-IDS2018, UNSW-NB15	**F1:** 98.00% (NSL), 97.05% (CSE), 90.45% (UNSW)
[52]	2024	UNSW-NB15, CIC-IDS2017, NSL-KDD	**Acc:** 99.21% **F1:** 99.00%

Table 2.1

Research Paper Name	**Release Date**	**Dataset Used**	**Result Metrics**
[61]	2024	NSL-KDD, UNSW-NB15, CICIDS2017, CIC-AWS-2018	DL models (89%-99% Acc) outperform SIDS/AIDS
[53]	2025	CIC-IDS2017, UNSW-NB15	**Acc:** 99.48% **F1:** 98.46% **Text Gen Acc:** 95.53%
[62]	2025	X-IIoTID, WUSTL-IIoT, Edge-IIoTset, MQTTset	**Avg F1:** 96.79%
[55]	2026	CSE-CIC-IDS2018	**Test Acc:** 96.94%
[63]	2026	ToN IoT	**AUC-ROC:** 0.994 **F1:** 0.976 **AUC-PR:** 0.956

Part IV

General Conclusion

This thesis presented the **Hybrid Masked Spatio-Temporal Transformer (HM-STT)**, a novel architecture for NIDS. By treating network flows as spatio-temporal volumes and utilizing tubelet embeddings, our model learns robust representations without labor-intensive labeling. Our analysis of SAFE (2025) and Falcon (2021) highlights the industry's trajectory towards self-supervision. HM-STT contributes by offering a unified model that respects the intrinsic structure of network data. Future work will focus on optimizing final efficiency for edge deployment and integrating GNNs to model inter-flow correlations.

Bibliography

[1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new dataset in recent network intrusion detection," *ICISSP*, pp. 446–453, 2018.

[2] R. Jordaney *et al.*, "Transcend: Detecting concept drift in malware classifiers," in *USENIX Security Symposium*, 2017.

[3] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Communications surveys & tutorials*, vol. 18, no. 2, pp. 1153–1176, 2016.

[4] J. Liu *et al.*, "Self-supervised learning for network intrusion detection: A survey," *IEEE Communications Surveys & Tutorials*, 2024.

[5] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM computing surveys (CSUR)*, vol. 46, no. 4, pp. 1–37, 2014.

[6] A. Vaswani, N. Shazeer, N. Parmar, *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[7] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 16 000–16 009.

[8] X. Lin *et al.*, "Et-bert: A contextualized protocol representation learning framework for encrypted traffic classification," *The Web Conference (WWW)*, 2022.

[9] M. Luis-Bisbe *et al.*, "No pictures, please: On the efficiency of image-based network intrusion detection," *IEEE Transactions on Information Forensics and Security*, 2024.

[10] E. Aksan, M. Kaufmann, and O. Hilliges, "A spatio-temporal transformer for 3d human motion prediction," *arXiv preprint arXiv:2004.08692*, 2020.

[11] W. Chen, F. Wang, and H. Sun, "S2tnet: Spatio-temporal transformer networks for trajectory prediction in autonomous driving," in *Asian Conference on Machine Learning (ACML)*, vol. 157, 2021, pp. 454–469.

[12] N. Mahmood, N. Ghorbani, N. F. Troje, G. Pons-Moll, and M. J. Black, "Amass: Archive of motion capture as surface shapes," in *The IEEE International Conference on Computer Vision (ICCV)*, 2019. [Online]. Available: https://amass.is.tue.mpg.de.

[13] E. Luis-Bisbé, V. Morales-Gómez, D. Perdices, and J. E. López de Vergara, "No pictures, please: Using explainable artificial intelligence to demystify cnns for encrypted network packet classification," *Applied Sciences*, vol. 14, no. 13, p. 5466, 2024.

[14] G. Bertasius, H. Wang, and L. Torresani, "Is space-time attention all you need for video understanding?" In *International Conference on Machine Learning (ICML)*, 2021, pp. 813–824.

[15] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of intrusion detection systems: Techniques, datasets and challenges," *Cybersecurity*, vol. 2, no. 1, pp. 1–22, 2019.

[16] L. Bilge and T. Dumitra, "Before we knew it: An empirical study of zero-day attacks in the real world," in *Proceedings of the 2012 ACM conference on Computer and communications security*, ACM, 2012, pp. 833–844.

[17] P. Szor, *The art of computer virus research and defense*. Pearson Education, 2005.

[18] B. I. Hairab, M. S. Elsayed, A. D. Jurcut, and M. A. Azer, "Anomaly detection based on cnn and regularization techniques against zero-day attacks in iot networks," *IEEE Access*, vol. 10, pp. 98 427–98 440, 2022.

[19] J. Smith and A. Doe, "A literature review on security in the internet of things: Identifying and analysing critical categories," *MDPI Sensors*, vol. 25, no. 3, p. 1042, 2025.

[20] R. Johnson and K. Lee, "Overview on intrusion detection systems for computers networking security," *MDPI Electronics*, vol. 14, no. 2, p. 215, 2025.

[21] A. Alshamrani, S. Myneni, A. Chowdhary, and D. Huang, "A survey on advanced persistent threats: Techniques, solutions, challenges, and research opportunities," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 2, pp. 1851–1877, 2019.

[22] I. J. King and H. H. Huang, "Euler: Detecting network lateral movement via scalable temporal link prediction," *ACM Transactions on Privacy and Security*, vol. 26, no. 3, pp. 1–36, 2023.

[23] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero trust architecture," National Institute of Standards and Technology, Tech. Rep. NIST Special Publication (SP) 800-207, 2020.

[24] Z. Ahmad, A. S. Shahid, and R. Thurasamy, "Network intrusion detection system: A systematic study of machine learning and deep learning approaches," *Transactions on Emerging Telecommunications Technologies*, vol. 32, no. 1, e4150, 2021.

[25] M. Roesch *et al.*, "Snort: Lightweight intrusion detection for networks.," in *LISA*, vol. 99, 1999, pp. 229–238.

[26] D. E. Denning, "An intrusion-detection model," *IEEE Transactions on software engineering*, no. 2, pp. 222–232, 1987.

[27] B. Anderson and D. McGrew, "Machine learning for encrypted malware traffic classification: Accounting for noisy labels and non-stationarity," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 1725–1734.

[28] S. Axelsson, "The base-rate fallacy and the difficulty of intrusion detection," *ACM Transactions on Information and System Security (TISSEC)*, vol. 3, no. 3, pp. 186–205, 2000.

[29] E. M. Hutchins, M. J. Cloppert, and R. M. Amin, "Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains," *Leading Issues in Information Warfare & Security Research*, vol. 1, no. 1, p. 80, 2011.

[30] B. Biggio and F. Roli, "Wild patterns: Ten years after the rise of adversarial machine learning," *Pattern Recognition*, vol. 84, pp. 317–331, 2018.

[31] A. Dosovitskiy *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," *ICLR*, 2021.

[32] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2019. [Online]. Available: https://arxiv.org/abs/1810.04805.

[33] R. Balestriero, M. Ibrahim, V. Sobal, A. Morcos, D. Bouchacourt, *et al.*, "A cookbook of self-supervised learning," *arXiv preprint arXiv:2304.12210*, 2023.

[34] J. Xie, W. Li, X. Zhan, Z. Liu, Y.-S. Ong, and C. C. Loy, "Masked frequency modeling for self-supervised visual pre-training," in *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: https://arxiv.org/abs/2206.07706.

[35] S. Stolfo, W. Fan, W. Lee, A. Prodromidis, and P. Chan, *KDD Cup 1999 Data*, 1999.

[36] M. Tavallaee, E. Bagheri, W. Lu, and A. Ghorbani, "A detailed analysis of the kdd cup 99 data set," in *Second IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*, 2009.

[37] R. Panigrahi, *Cicids2017*, 2025. DOI: 10.21227/akxq-9v09. [Online]. Available: https://dx.doi.org/10.21227/akxq-9v09.

[38] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, 2018.

[39] N. Moustafa and J. Slay, "Unsw-nb15: A comprehensive data set for network intrusion detection systems (unsw-nb15 network data set)," in *Military Communications and Information Systems Conference (MilCIS)*, IEEE, 2015.

[40] K. Chris, *Database normalization – normal forms 1nf 2nf 3nf table examples*, freeCodeCamp, 2022.

[41] P. Xu, C. Eckert, and A. Zarras, "Falcon: Malware detection and categorization with network traffic images," in *International Conference on Artificial Neural Networks (ICANN)*, Springer, 2021, pp. 117–128.

[42] L. Mohammadpour, T. C. Ling, C. S. Liew, and A. Aryanfar, "A survey of cnn-based network intrusion detection," *Applied Sciences*, vol. 12, no. 16, p. 8162, 2022. DOI: 10.3390/app12168162.

[43] T. Kim and W. Pak, "Deep learning-based network intrusion detection using multiple image transformers," *Applied Sciences*, vol. 13, no. 5, p. 2754, 2023. DOI: 10.3390/app13052754.

[44] K. He, W. Zhang, X. Zong, and L. Lian, "Network intrusion detection based on feature image and deformable vision transformer classification," *IEEE Access*, vol. 12, pp. 44 335–44 350, 2024. DOI: 10.1109/ACCESS.2024.3376434.

[45] Canadian Institute for Cybersecurity, *Intrusion detection evaluation dataset (cicids2017)*, Accessed: 15 June 2018, 2017. [Online]. Available: http://www.unb.ca/cic/datasets/ids-2017.html.

[46] R. A. Abed, E. K. Hamza, and A. J. Humaidi, "A modified cnn-ids model for enhancing the efficacy of intrusion detection system," *Measurement: Sensors*, vol. 35, p. 101 299, 2024. DOI: 10.1016/j.measen.2024.101299.

[47] X. Liu, X. Fu, F. Huang, and L. Zhang, "Mean masked autoencoder with flow-mixing for encrypted traffic classification," *arXiv preprint arXiv:2603.29537*, 2026. [Online]. Available: https://arxiv.org/abs/2603.29537.

[48] A. K and A. John, "A deep learning based intrusion detection system using transformers," in *Preprint / Technical Report*, 2022.

[49] Z. Wu, H. Zhang, P. Wang, and Z. Sun, "Rtids: A robust transformer-based approach for intrusion detection system," *IEEE Access*, vol. 10, pp. 64 375–64 387, 2022. DOI: 10.1109/ACCESS.2022.3182333.

[50] Y. Liu and L. Wu, "Intrusion detection model based on improved transformer," *Applied Sciences*, vol. 13, no. 10, p. 6251, 2023. DOI: 10.3390/app13106251.

[51] L. D. Manocchio, S. Layeghy, W. W. Lo, G. K. Kulatilleke, M. Sarhan, and M. Portmann, "Flowtransformer: A transformer framework for flow-based network intrusion detection systems," *Expert Systems with Applications*, vol. 241, p. 122 564, 2024. DOI: 10.1016/j.eswa.2023.122564.

[52] F. Ullah, S. Ullah, G. Srivastava, and J. C.-W. Lin, "Ids-int: Intrusion detection system using transformer-based transfer learning for imbalanced network traffic," *Digital Communications and Networks*, vol. 10, pp. 190–204, 2024. DOI: 10.1016/j.dcan.2023.03.008.

[53] P. Rajkumardheivanayahi, R. Berry, N. U. Costagliola, L. Fiondella, N. D. Bastian, and G. Kul, "Explainability of network intrusion detection using transformers: A packet-level approach," *IEEE Access*, vol. 13, pp. 5153–5174, 2025. DOI: 10.1109/ACCESS.2024.3525008.

[54] E. Caville, W. W. Lo, S. Layeghy, and M. Portmann, "Anomal-e: A self-supervised network intrusion detection system based on graph neural networks," *arXiv preprint arXiv:2207.06819*, 2023. [Online]. Available: https://doi.org/10.48550/arXiv.2207.06819.

[55] P. Appiahene, S. O. Berchie, E. Botchway, *et al.*, "Network intrusion detection using a hybrid graph-based convolutional network and transformer architecture," *PLoS One*, vol. 21, no. 1, e0340997, 2026. DOI: 10.1371/journal.pone.0340997.

[56] H. Hindy, R. Atkinson, C. Tachtatzis, J.-N. Colin, E. Bayne, and X. Bellekens, "Utilising deep learning techniques for effective zero-day attack detection," *Electronics*, vol. 9, no. 10, p. 1684, 2020. DOI: 10.3390/electronics9101684. [Online]. Available: https://doi.org/10.3390/electronics9101684.

[57] M. Sarhan, S. Layeghy, M. Gallagher, and M. Portmann, *From zero-shot machine learning to zero-day attack detection*, 2021. [Online]. Available: https://doi.org/10.1007/s10207-023-00676-0.

[58] V. Kumar and D. Sinha, "A robust intelligent zero-day cyber-attack detection technique," *Complex & Intelligent Systems*, vol. 7, pp. 2211–2234, 2021. DOI: 10.1007/s40747-021-00396-9. [Online]. Available: https://doi.org/10.1007/s40747-021-00396-9.

[59] B. I. Hairab, H. K. Aslan, M. S. Elsayed, A. D. Jurcut, and M. A. Azer, "Anomaly detection of zero-day attacks based on cnn and regularization techniques," *Electronics*, vol. 12, no. 3, p. 573, 2023. DOI: 10.3390/electronics12030573. [Online]. Available: https://doi.org/10.3390/electronics12030573.

[60] C. Appani and D. P. Guda, "Self-supervised representation learning for zero-day attack detection in encrypted network traffic," *Computer Fraud and Security*, 2023, ISSN: 1873-7056.

[61] A. Armijos and E. Cuenca, *Zero-day attacks: Review of the methods used based on intrusion detection and prevention systems*, 2024.

[62] E. Li, Z. Shang, O. Gungor, and T. Rosing, "Safe: Self-supervised anomaly detection framework for intrusion detection," *arXiv preprint arXiv:2502.07119*, 2025.

[63] S. Nayak, *Threatformer-ids: Robust transformer intrusion detection with zero-day generalization and explainable attribution*, Incedo Inc, USA, 2026. [Online]. Available: https://doi.org/10.48550/arXiv.2603.00185.